

Prescription App — Frontend UI + Image Storage + Patient Details Extraction

1. Backend Improvements

Update Prescription Model

File:

```
backend/models/Prescription.js
```

Replace with:

```
const mongoose = require('mongoose');

const medicineSchema = new mongoose.Schema({
  name: String,
  dosage: String,
  timing: String,
  raw: String
});

const prescriptionSchema = new mongoose.Schema({
  patientName: String,
  doctorName: String,
  hospitalName: String,
  followUpDate: String,

  imagePath: String,

  text: String,

  medicines: [medicineSchema],

  createdAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('Prescription', prescriptionSchema);
```

Update Upload Route

File:

```
backend/routes/upload.js
```

Replace this section:

```
const savedPrescription = await Prescription.create({
  text,
  medicines: uniqueMedicines
});
```

WITH:

```
// Extract patient name
const patientNameMatch = text.match(/Name[:\s]+([A-Za-z ]+)/i);
const patientName = patientNameMatch
  ? patientNameMatch[1].trim()
  : 'Unknown';

// Extract hospital name
const hospitalMatch = text.match(/Asian Institute of Medical Sciences/i);
const hospitalName = hospitalMatch
  ? hospitalMatch[0]
  : 'Unknown Hospital';

// Extract follow-up date
const followUpMatch = text.match(/FOLLOW UP ADVICE[:\s]+([^\n]+)/i);
const followUpDate = followUpMatch
  ? followUpMatch[1].trim()
  : null;

// Extract doctor name (basic sample)
const doctorName = 'Doctor';

const savedPrescription = await Prescription.create({
  patientName,
  doctorName,
  hospitalName,
  followUpDate,

  imagePath: req.file.path,

  text,
```

```
    medicines: uniqueMedicines
  });
```

Add API to Get Prescriptions

Add BEFORE:

```
module.exports = router;
```

Add:

```
router.get('/all', async (req, res) => {
  try {
    const prescriptions = await Prescription.find()
      .sort({ createdAt: -1 });

    res.json({
      success: true,
      data: prescriptions
    });

  } catch (error) {
    res.status(500).json({
      error: error.message
    });
  }
});
```

Restart Backend

```
cd backend
node server.js
```

2. React Frontend

Create React App

Open new terminal:

```
cd prescription-app  
npx create-react-app frontend
```

Install Axios

```
cd frontend  
npm install axios
```

Replace App.js

File:

```
frontend/src/App.js
```

Replace everything with:

```
import React, { useEffect, useState } from 'react';  
import axios from 'axios';  
  
function App() {  
  const [file, setFile] = useState(null);  
  const [prescriptions, setPrescriptions] = useState([]);  
  const [loading, setLoading] = useState(false);  
  
  const fetchPrescriptions = async () => {  
    try {  
      const res = await axios.get(  
        'http://localhost:5001/api/upload/all'  
      );  
  
      setPrescriptions(res.data.data);  
    } catch (error) {  
      console.log(error);  
    }  
  };  
};
```

```

useEffect(() => {
  fetchPrescriptions();
}, []);

const uploadFile = async () => {
  if (!file) {
    alert('Select file');
    return;
  }

  const formData = new FormData();
  formData.append('file', file);

  try {
    setLoading(true);

    await axios.post(
      'http://localhost:5001/api/upload',
      formData
    );

    alert('Prescription uploaded');

    fetchPrescriptions();

    setLoading(false);
  } catch (error) {
    console.log(error);
    setLoading(false);
  }
};

return (
  <div style={{ padding: 20 }}>

    <h1>Prescription Medicine Tracker</h1>

    <input
      type="file"
      onChange={(e) => setFile(e.target.files[0])}
    />

    <button
      onClick={uploadFile}
      style={{
        marginLeft: 10,
        padding: 10
      }}
    >
      {loading ? 'Uploading...' : 'Upload Prescription'}
  </div>
);

```

```

</button>

<hr />

<h2>Saved Prescriptions</h2>

{
  prescriptions.map((prescription) => (
    <div
      key={prescription._id}
      style={{
        border: '1px solid #ccc',
        padding: 20,
        marginBottom: 20,
        borderRadius: 10
      }}
    >

      <h3>
        Patient: {prescription.patientName}
      </h3>

      <p>
        Hospital: {prescription.hospitalName}
      </p>

      <p>
        Follow-up: {prescription.followUpDate}
      </p>

      <h4>Medicines</h4>

      {
        prescription.medicines.map((med, index) => (
          <div
            key={index}
            style={{
              background: '#f4f4f4',
              padding: 10,
              marginBottom: 10,
              borderRadius: 6
            }}
          >
            <strong>{med.name}</strong>

            <p>
              Dosage: {med.dosage || 'N/A'}
            </p>

            <p>
              Timing: {med.timing || 'N/A'}

```

```
        </p>
      </div>
    ))
  }
</div>
))
}
</div>
);
}

export default App;
```

Start React Frontend

```
npm start
```

App URLs

Frontend:

```
http://localhost:3000
```

Backend:

```
http://localhost:5001
```

Final Features

You now have:

- Prescription image upload
- Google OCR extraction
- Medicine parsing
- MongoDB storage
- Patient details extraction
- Image path storage
- React frontend UI

- Medicine cards
 - Prescription history
-

Recommended Next Features

Add Medicine Reminders

Use:

- node-cron
 - Firebase notifications
 - Twilio WhatsApp reminders
-

Add Authentication

Use:

- JWT
 - Login/Register pages
-

Add Dashboard

Features:

- total medicines
 - missed doses
 - follow-up reminders
 - doctor visit tracking
-

Deploy Free

Frontend:

- Vercel

Backend:

- Render

Database:

- MongoDB Atlas